

Lab 4 - Hierarchical Prompting for RTL Generation

Example used: PWM Generator

Models compared: OpenAI ChatGPT (GPT-4o-mini) and DeepSeek (NVIDIA Integrate API)

1. Abstract

This report evaluates *hierarchical prompting* for hardware code generation using a PWM generator as the running example. We compare two LLMs (ChatGPT vs. DeepSeek), contrast hierarchical prompting with flat prompting (AutoChip-style), and document debugging practices discovered during the lab. Hierarchical prompting successfully produced working RTL with self-checking testbenches, with differences observed in iteration counts, verbosity, and failure modes.

2. Background & Goals

- **Flat prompting (baseline):** Request a top-level module in one shot and compile/simulate.
- **Hierarchical prompting (this lab):** Generate leaf modules first, then integrate into a top-level wrapper. Iterate with feedback from compiler/simulator.
- **Goals:**
 - 1) Build a parameterizable PWM using hierarchical prompting.
 - 2) Run the same design through two different LLMs.
 - 3) Compare results to flat prompting.

3. Experimental Setup

3.1 Toolchain

- **Synthesis/simulation:** Icarus Verilog (iverilog + vvp).
- **Notebook runtime:** Google Colab.
- **Pass criterion:** Testbench prints the token passed!.
- **Compile flags:** -g2012 to accept SystemVerilog syntax when needed (we enforced pure Verilog in prompts but left flag on for safety).
- **Logging:** Console logs per iteration were written to text files for each stage; timings were recorded for total generation and error handling.

3.2 Models

- **ChatGPT (OpenAI):** OpenAI chat completions API (model family GPT).
- **DeepSeek (NVIDIA Integrate API):** OpenAI-compatible endpoint; used deepseek-ai/deepseek-r1 via `base_url=https://integrate.api.nvidia.com/v1`.

3.3 Prompt Style (per model)

Each run starts with a strict instruction to **output Verilog code only**, no prose, then the **module-specific spec**. Errors and warnings from tool runs are appended in subsequent iterations so the model can fix issues.

4. PWM Design Specification

Module name: `top_module`

Parameters: `WIDTH=16`

Inputs: `clk`, `rst_n`, `enable`, `period[WIDTH-1:0]`, `duty[WIDTH-1:0]`

Outputs: `pwm_out`, `tick`

Behavior: - Synchronous reset clears counter and outputs.

- When `enable=1`, the counter increments each cycle; wraps at `period-1` and asserts `tick` for one cycle.

- `pwm_out = 1` while `count < min(duty, period)`; otherwise 0 (clamps `duty ≥ period` to 100%).

5. Hierarchical Prompting Plan

We decomposed the PWM into: 1) **pwm_counter** — leaf, sequential counter with wrap/tick.

2) **pwm_compare** — leaf, combinational comparator `count < min(duty, period)`.

3) **top_module** — integrates counter+comparator and forwards ports.

File/Testbench convention used by the loop: - Stage *i* compiles `./<module>.v` with `./<module>tb.v`.

- For the final stage we used a comprehensive top-level TB (`top_moduletb.v`) that produces the passed! token on success.

6. Workflow

The steps below match the exact workflow I used in Colab.

Step 1 - Write the three testbenches

- `pwm_countertb.v` — checks wrap and tick behavior.
- `pwm_comparetb.v` — sweeps `count` vs. `min(duty, period)` and validates `pwm_out`.

- `top_moduletb.v` — end-to-end functional checks (0%, 25%, 50%, 75%, 100%, clamp, and dynamic period change).
Important: Ensure these files live in the current working directory so `hier_gen` finds `./<module>tb.v`.

Step 2 - Define submodules

```
submodules = [  
    ["pwm_counter", "synchronous up-counter with wrap tick for PWM",  
     "input clk, input rst_n, input enable, input [WIDTH-1:0] period, output reg [WIDTH-1:0] count, output reg tick"],  
    ["pwm_compare", "combinational comparator that outputs 1 when count < min(duty, period)",  
     "input [WIDTH-1:0] count, input [WIDTH-1:0] duty, input [WIDTH-1:0] period, output wire pwm_out"],  
    ["top_module", "parameterizable PWM generator top integrating counter+comparator",  
     "input wire clk, input wire rst_n, input wire enable, input wire [WIDTH-1:0] period, input wire [WIDTH-1:0] duty, output wire pwm_out, output wire tick"]  
]
```

Step 3 - Run `hier_gen(submodules)`

This calls the feedback loop per stage: 1. Generate RTL from the spec.

2. Compile with `iverilog`.

3. If compile/sim fails, append the error back into the conversation and regenerate.

4. Stop when the TB prints passed! or `max_iterations` is reached.

Step 4 - Common pitfalls & fixes

- **SystemVerilog vs Verilog:** Use `-g2012` for safety, but prompt the model to output *pure Verilog-2001*.
- **Missing modules:** Ensure leaf files are available when building the top, or embed prior modules in each iteration (the provided loop expects prior modules to be included in the generated file).
- **Pass token not detected:** Make the checker look for "passed!" anywhere in stdout or ensure it's the last printed line.
- **Excess commentary:** Prepend a strict no-prose instruction; additionally, strip content outside the first module..endmodule as a safety net.

7. Results

7.1 ChatGPT (GPT-4o-mini)

- Early leaf stages compiled and ran successfully.
- The top-level initially failed with **unknown module** and **port mismatch** errors (tick not found), which the loop resolved after a few iterations.

- Final top-level testbench reported success and printed timings.

Observed characteristics: concise code, fewer syntactic issues; moderate iterations needed at the integration step due to interface alignment.

7.2 DeepSeek (NVIDIA Integrate)

- Early stages ran; the top-level had several syntax and binding errors at first (unknown module, port name mismatch, malformed statements).
- The loop eventually converged, with successful final simulation and timings printed.
- Tended to include extra explanatory prose unless constrained—mitigated by stricter instruction and response filtering.

Observed characteristics: more verbose outputs and occasional syntax detours; still converged under feedback

7.3 Timing/Iteration snapshots

- Both models reached “**Testbench ran successfully**” for the final top module after passing through error-fix iterations.
- GPT generally produced briefer logs; DeepSeek spent more time in generation while still converging.

8. Comparison: Hierarchical vs Flat Prompting

Flat prompting (AutoChip baseline): In prior work, PWM and other modules often passed on the first try with 0 mismatches across ~2400 PWM samples, using a simple single-shot prompt and a feedback loop that rarely needed more than one iteration. This shows flat prompting can be extremely effective on simpler designs.

Hierarchical prompting (this lab): - Pros: encourages cleaner decomposition, reusable leaf designs, and clearer error localization (e.g., counter vs comparator bugs).
- Cons: added orchestration (TB per stage), interface drift between stages, potential for “unknown module” errors and partial integration failures that require careful prompts and directory/testbench handling.

Takeaway: For small/medium designs like PWM, both approaches work; hierarchical offers educational clarity and better scalability to larger hierarchies, while flat prompting may be faster when the spec is simple and the model is strong.

9. Lessons Learned & Debugging Playbook

- 1) **Compiler mode:** If you see syntax error near typedef/logic, you're likely compiling SV as Verilog—add `-g2012` or remove SV constructs.
- 2) **Module discovery:** Unknown module type; either the leaf file isn't included or the model failed to include prior modules in the generated file; re-prompt to include prior leafs or add library search paths.
- 3) **Port mismatches:** Keep a single source of truth for the interface; auto-check TB vs RTL port lists.
- 4) **Pass detection:** Search for passed! anywhere in stdout, or print it last.
- 5) **No-prose outputs:** Use a strict instruction and trim to `module...endmodule`.
- 6) **Surface errors early:** Print `iverilog/vvp stderr/stdout` directly for quick diagnosis.

10. Conclusion

Hierarchical prompting successfully produced a correct, parameterizable PWM in Verilog. While it required more iteration than flat prompting, it clarified responsibilities between submodules and made debugging more systematic. Across models, GPT converged with fewer syntactic issues; DeepSeek converged reliably with stronger constraints. For larger RTL hierarchies, the hierarchical approach is preferable; for small leaf-like blocks, flat prompting remains highly efficient.